Slovak Technical University in Bratislava

Faculty of Informatics and Information Technologies

Developing Neuron Network Architectures for Sports Image Classification

Introduction to Neural Networks

Filip Brutovský
Anastasiia Rerikh

Practice supervisor: Ing. Matej Halinkovič
Time of the practice: Monday 15:00 - 16:50

Filip Brutovský
Anastasiia Rerikh

Table of Contents

Introduction **3**
Project Overview 3
Project's Structure 3
Dataset Information 4
Datasplit: 4
Work Distribution 4
Libraries used: 4
Deep Learning and Scientific Computing: 4
Machine Learning and Feature Engineering: 4
Computer Vision: 4
Data Processing and Visualisation: 5
Model Persistence and Configuration: 5
Experiment Tracking: 5
Data Preprocessing **6**
Initial Data Exploration 6
Dataset Statistics 6
Classes and Their Distribution 6
Class Distribution Table: 7
Class Balance Analysis: 7
Data Quality Assessment 7
Size Verification: 7
Corruption Detection: 8
Blank Image Detection: 8
Duplicate Detection: 8
Mislabeled images: 8
Potential issues: 8
Identified Challenges: 8
Mitigation Strategies: 8
Features Variable Encoding 9
Feature Extraction and Selection **10**
Extracted Features 10
1. Colour Histogram 10
2. Histogram of Oriented Gradients (HOG) 10
3. Local Binary Patterns (LBP) 11
4. ORB (Oriented FAST and Rotated BRIEF) 12
Feature Selection Process 12
Variance Filtering (with threshold 1e-5) 12
Mutual Information Analysis 13
Top 10 Features by Mutual Information: 13
Dimensionality Reduction (PCA) 14

Correlation Analysis 15
Feature Handling in CNN-Based Models (SCNN, DCNN, ResNet) 15
Conclusion for Feature Selection and Extraction 16
Neural Network Overview 17
Model Architectures: 17
Model 0: SimpleCNN (SCNN) 17
Model 1: DeepCNN (DCNN) 17
Model 2: ResNet 18
Model 3: FENN (Feature Extraction Neural Network) 18
Short Architecture Comparison 18
Training Configuration 19
Hyperparameters: 19
Global Configuration 19
Optimisation Parameters 19
Regularisation Parameters 19
Training Modes: 19
Simple Training (Mode 1) 19
Complex Training (Mode 0) 19
Training Pipeline: 20
Experiments 21
Concluded Experiments: 21
Results and evaluation 24
Best Network Performance 24
Final Validation Metrics (for the best) 24
Confusion Matrix 24
Interpretation: 24
Best FENN Performance 25
Final Validation Metrics (for the best) 25
Confusion Matrix 25
Interpretation: 25
Comparative Analysis 26
Graphs 26
Experimental conclusion 29
Overall Conclusion 30

Filip Brutovský
Anastasiia Rerikh

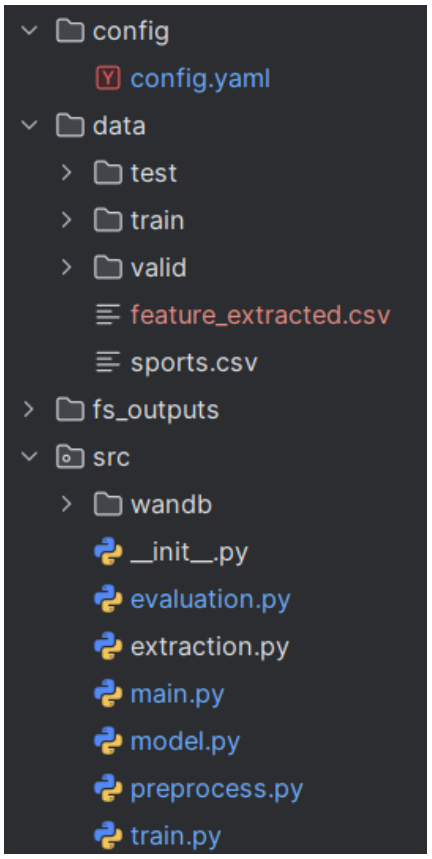
Introduction

Project Overview

This project implements a multi-class image classification system for sports recognition using deep learning and traditional computer vision techniques. The system employs four different neural network architectures to classify sports activities from images, providing a comparative analysis of different approaches ranging from simple CNNs to feature-engineered neural networks.

The objective of our project is to develop and compare multiple neural network architectures for accurate sports classification from image data.

Project's Structure



Where:

- config/config.yaml - Configuration
- data/test - Test images
- data/train - Training images
- data/valid - Validation images
- data/sports.csv - Made by us dataset
- data/feature_extracted.csv - dataset of extracted features
- fs_outputs - outputs from feature selection
- src/wandb - directory with conducted experiments
- src/evaluation - Evaluation metrics and visualisation
- src/extraction - Feature extraction methods
- src/main - Main execution script
- src/model - Neural network architectures
- scr/prepropcess - Data preprocessing pipeline
- scr/train - Training procedures

Filip Brutovský
Anastasiia Rerikh

Dataset Information

Dataset Name: Sports Image Classification Dataset

Total Images: 2077

Image Dimensions: 224×224 pixels (RGB)

Number of Classes: 14

Datasplit:

Training Set: 150~200 images

Validation Set: 5 images

Test Set: 5 images

Work Distribution

Filip Brutovský (xbrutovskyf@stuba.sk): 50%

Anastasiia Rerikh (xrerikh@stuba.sk): 50%

Libraries used:

Deep Learning and Scientific Computing:

- PyTorch (torch, torchvision) - Neural network implementation and training
- NumPy - Numerical operations and array manipulation

Machine Learning and Feature Engineering:

- scikit-learn - Feature selection, preprocessing, and evaluation metrics
- scikit-image - Image feature extraction (HOG, LBP)

Computer Vision:

- OpenCV (cv2) - Image processing and feature extraction
- PIL (Python Imaging Library) - Image loading and manipulation

Data Processing and Visualisation:

- Pandas - Data manipulation and CSV handling
- Matplotlib - Plotting and visualisation
- Seaborn - Statistical data visualisation

Model Persistence and Configuration:

- joblib - Model serialisation
- PyYAML - Configuration file parsing

Experiment Tracking:

- Weights & Biases (wandb) - Experiment logging and tracking

Filip Brutovský
Anastasiia Rerikh

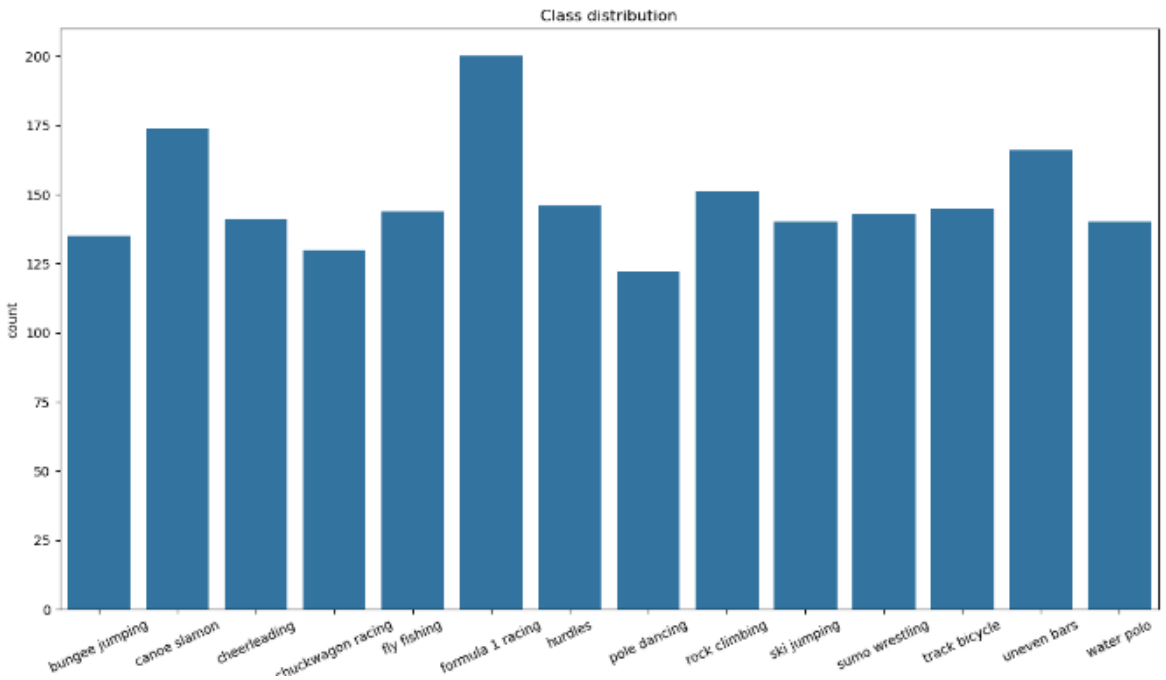
Data Preprocessing

Initial Data Exploration

Dataset Statistics

Metrics	**Values**
Total Images	2077
Unique Classes	14
Average Images per Class	148
Min Images per Class	122 (Pole Dancing)
Max Images per Class	200 (Formula 1 Racing)

Classes and Their Distribution



Class Distribution Table:

Sport	Train Count	Valid Count	Test Count	Total	Percentage
Sumo Wrestling	133	5	5	143	6.884930
Water Polo	130	5	5	140	6.740491
Ski Jumping	130	5	5	140	6.740491
Chuckwagon Racing	120	5	5	130	6.259027
Cheerleading	131	5	5	141	6.788637
Canoe Slalom	164	5	5	174	8.377468
Formula 1 Racing	190	5	5	200	9.629273
Fly Fishing	134	5	5	144	6.933077
Bungee Jumping	125	5	5	135	6.499759
Rock Climbing	141	5	5	151	7.270101
Pole Dancing	112	5	5	122	5.873857
Track Bicycle	135	5	5	145	6.981223
Hurdles	136	5	5	146	7.029369
Uneven Bars	156	5	5	166	7.992297

Class Balance Analysis:

Most represented class: Formula 1 racing (9.63%)

Least represented class: pole dancing (5.87%)

Imbalance ratio: 1.64:1 (max/min ratio)

Conclusion: The dataset shows mild class imbalance with a ratio of 1.64:1. This level of imbalance is generally manageable and should not severely impact model performance.

Data Quality Assessment

Size Verification:

Target size: 224×224 pixels

Non-conforming images: 0 (all images are already 224×224)

Corruption Detection:

Corrupt/Unreadable images found: 0.

Blank Image Detection:

Threshold: Standard deviation < 5

Near-blank images detected: 0.

Duplicate Detection:

Duplicate rows in metadata: 0.

Mislabeled images:

None.

Potential issues:

Identified Challenges:

- **Intra-class Variability:** Sports images captured from different angles, lighting conditions, and distances.
- **Inter-class Similarity:** Some sports share visual characteristics:
  - Water sports (Water Polo, Canoe Slalom) both feature water backgrounds
  - Gymnastic events (Uneven Bars, Pole Dancing) both involve vertical apparatus
  - Track events (Hurdles, Track Bicycle) share similar venue aesthetics
  - Outdoor adventure sports (Rock Climbing, Bungee Jumping) have similar natural backgrounds
- **Background Noise:** Varying backgrounds that may confuse the model
- **Scale Variation:** Athletes/equipment at different scales within images

Mitigation Strategies:

1. **Data augmentation** to increase training sample diversity:
   - Random Horizontal Flip (probability: 0.3) to increase viewpoint diversity.
   - Random Rotation (±15 degrees) to account for camera angle variations.
   - Colour Jitter (brightness: 0.2, contrast: 0.2, saturation: 0.2, hue: 0.1) to simulate different lighting conditions

Applied only to the training set.

2. **Normalisation** to handle lighting variations is performed using the computed mean and standard deviation from the training set and is applied to all CNN models (SCNN, DCNN, ResNet).
3. **Feature engineering** (feature extraction) to capture discriminative patterns:
    - Colour Histogram (8×8×8 bins across RGB)
    - HOG features (9 orientations, 16×16 pixels per cell)
    - LBP texture features
    - ORB keypoint descriptors (500 features)

Applied in the FENN model.

4. Feature **scaling** and **dimensionality reduction**:
    - StandardScaler for feature normalisation
    - Variance filtering (threshold: 1e-5)
    - PCA for dimensionality reduction (95% variance retention) for FENN.

Features Variable Encoding

Original labels were string class names, so we encoded them as integers from 0 to 13 in alphabetical order of their names. The result is saved to *fs_outputs/label_encoder.joblib*, which is used later for inverse transformation during evaluation.

Feature Extraction and Selection

The Feature Extraction Neural Network (FENN) uses traditional computer vision techniques to extract meaningful features from images before neural network processing. It is less automated and more manual, similar to how it has been done in the past.

Extracted Features

1. Colour Histogram

- **Method:** 3D histogram across RGB channels
- **Bins:** 8×8×8 = 512 bins
- **Range:** [0, 256] per channel
- **Normalisation:** L2 normalisation
- **Output Dimension:** 512 features

Colour Histogram creates a compact representation of the colour distribution in an image by dividing the RGB colour space into 512 distinct bins and counting how many pixels fall into each bin. This essentially creates a "fingerprint" of the image's colour composition.

Different sports have characteristic colour profiles - for example, water polo images contain significant amounts of blue (water), ski jumping has white (snow) and blue (sky), while Formula 1 racing features the vibrant colours of race cars and tracks. The histogram captures these distinctive colour patterns without being affected by the exact spatial arrangement of colours, making it robust to variations in camera angle or athlete positioning.

2. Histogram of Oriented Gradients (HOG)

- **Bins:** 9 bins
- **Pixels per Cell:** 16×16
- **Cells per Block:** 2×2
- **Block Normalisation:** L2-Hys

HOG analyses the direction and strength of edges in the image by computing gradients (changes in intensity) and grouping them into 9 directional bins. The image is divided into small cells, and for each cell, a histogram of gradient directions is computed.

These cells are then grouped into overlapping blocks for normalisation, capturing both local detail and broader structural information.

HOG is good at capturing the characteristic shapes and poses associated with different sports. For instance, the distinctive postures in pole dancing (vertical orientation), the crouched position in ski jumping, or the horizontal body alignment in hurdles create unique edge patterns. HOG is particularly effective because it's relatively invariant to small changes in lighting and can capture the overall "silhouette" of an activity, which is often the most discriminative feature between sports.

3. Local Binary Patterns (LBP)
    - **Points (P):** 8
    - **Radius (R):** 1
    - **Method:** Uniform patterns
    - **Histogram Bins:** 59 (uniform patterns + 1)
    - **Normalisation:** L1 normalisation
    - **Output Dimension:** 58 features

LBP encodes local texture by comparing each pixel with its 8 surrounding neighbours. If a neighbour is brighter, it gets a 1; if darker, a 0. This creates an 8-bit binary pattern for each pixel. The method focuses on "uniform" patterns (those with at most two 0-1 transitions) which represent fundamental texture primitives like edges, corners, and flat areas. These patterns are then histogrammed across the image.

Different sports environments and equipment create distinctive texture patterns. Water surfaces have characteristic ripple textures in water polo and canoe slalom, snow has a unique crystalline texture in ski jumping, the weave patterns of uniforms and mats differ between sports like cheerleading and sumo wrestling, and artificial surfaces (tracks, courts) have different textures than natural ones. LBP is computationally efficient, rotationally invariant, and particularly good at capturing these micro-textures that human eyes might not consciously notice but that distinguish different sports environments.

4. ORB (Oriented FAST and Rotated BRIEF)
    - **Features:** 500 keypoints maximum
    - **Descriptor Length:** 32 bytes per keypoint
    - **Aggregation Method:** Mean descriptor vector
    - **Output Dimension:** 32 features

ORB detects distinctive "keypoints" in the image - corners, edges, and other high-contrast features that are easy to relocate even if the image is rotated, scaled, or partially obscured. For each keypoint, it creates a 32-byte binary descriptor that captures the local image pattern around that point. Since the number of keypoints varies by image, we average all descriptors into a single 32-dimensional vector for consistency.

ORB captures distinctive equipment and environmental markers that are characteristic of specific sports. For example, the geometric patterns of hurdles, the distinctive shape of Formula 1 cars, the parallel bars in uneven bars gymnastics, or the bicycle frame geometry in track cycling. Unlike the other features that look at overall statistics, ORB identifies specific recurring visual elements. The averaging strategy makes it robust - even if individual images show equipment from different angles, the average descriptor captures what these objects generally "look like" across many examples. This is particularly valuable for sports with distinctive equipment or architectural features (racing tracks, climbing walls, ski jumps, etc.).

Feature Selection Process

Variance Filtering (with threshold 1e-5)

Variance Filtering acts as a first-pass filter to eliminate "useless" features before more expensive analysis. After extracting 6000+ features from colour histograms, HOG, LBP, and ORB descriptors, some features might be nearly constant across all images (e.g., a specific colour bin that no sports image uses, or a texture pattern that never appears). These provide no discriminative power between sports classes.

Results:

Feature count before Variance Filtering: 6686

Feature count after Variance Filtering: 6628

Filip Brutovský
Anastasiia Rerikh

Mutual Information Analysis

Mutual information analysis ranks each remaining feature by how much information it provides about the sport class. This helps us identify which extracted features are actually useful - for instance, we might discover that certain HOG orientations or specific colour histogram bins are particularly informative for distinguishing between sports, while others are redundant or noisy.

The result will be the top K (100 for our project) of the most informative features, balancing between retaining useful information and reducing dimensionality for faster training. The selected features are then saved to *fs_outputs/selected_mi_features.csv.*

Top 10 Features by Mutual Information:

Rank	Feature	MI Score	*Feature Type
1	f6598	0.344080	ORB
2	f6596	0.330388	ORB
3	f0	0.294591	Colour
4	f6605	0.260130	ORB
5	f229	0.250947	Colour
6	f84	0.247447	Colour
7	f1	0.242145	Colour
8	f9	0.234756	Colour
9	f156	0.234298	Colour
10	f428	0.230684	Colour

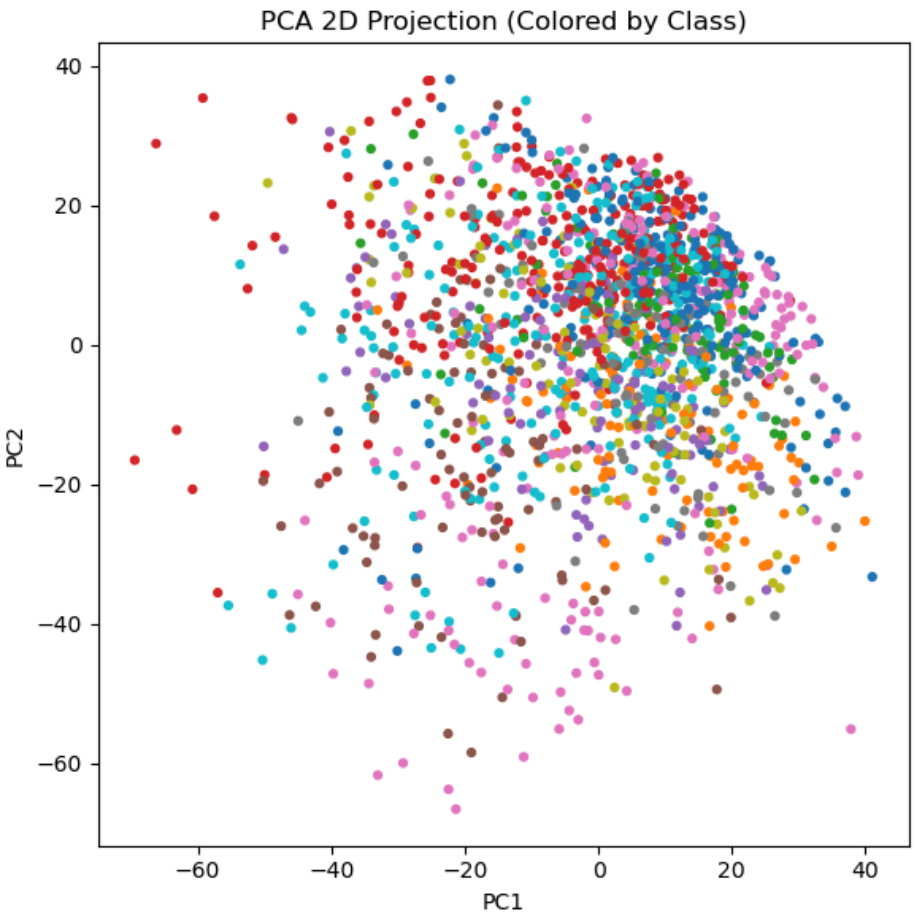
**Feature type: HOG/Colour/LBP/ORB*

Filip Brutovský
Anastasiia Rerikh

Dimensionality Reduction (PCA)

After variance filtering, we still have thousands of features that may be highly correlated (e.g., adjacent colour histogram bins, or similar HOG orientations). PCA transforms these into a smaller set of uncorrelated "principal components" that capture the most important patterns in the data. This dramatically reduces the input dimension to our neural network while preserving 95% of the information, making training faster and reducing overfitting risk.

95% variance retention has been chosen as it is the standard trade-off threshold. Components beyond 95% typically capture noise rather than signal. By keeping components that explain 95% the variance, we retain nearly all meaningful patterns while discarding the least important 5% that likely represents measurement noise, lighting variations, or other irrelevant factors.



Correlation Analysis

Correlation analysis examines relationships between features to identify redundancy. Highly correlated features ($|r| > 0.8$) provide similar information and can be candidates for removal.

For FENN Model: Since PCA already handles correlation by creating orthogonal (uncorrelated) components, explicit correlation-based filtering was not necessary for our FENN pipeline. PCA inherently addresses multicollinearity by transforming correlated features into independent principal components.

For CNN Models (SCNN, DCNN, ResNet): Correlation analysis is not applicable to these models (CNNs operate on raw pixel values, not engineered features).

Feature Handling in CNN-Based Models (SCNN, DCNN, ResNet)

Unlike the FENN model, which relies on manually engineered features, the CNN-based models (SCNN, DCNN, and ResNet) operate directly on raw image pixels. These models do not require explicit feature extraction or feature selection, as the convolutional layers automatically learn hierarchical visual representations from the data.

Through successive convolution and pooling operations, the CNNs progressively learn:

- Low-level features such as edges, corners, and colour gradients in early layers,
- Mid-level features such as object parts and textures in intermediate layers,
- High-level semantic patterns, such as full sports scenes and athlete poses in deeper layers.

Because CNNs learn features dynamically during training, traditional feature selection techniques such as variance filtering, mutual information ranking, or PCA are neither required nor applicable.

Filip Brutovský
Anastasiia Rerikh

Conclusion for Feature Selection and Extraction

The feature selection pipeline reduced the original 6686 engineered features to a compact and informative representation. Variance filtering removed low-information features, mutual information further selected the 100 most discriminative features, and PCA reduced the dimensionality to 894 components while preserving 95% of the total variance. This significantly lowered the computational complexity without substantial information loss. While this process was essential for the FENN model, CNN-based models operated directly on raw images and therefore did not require explicit feature selection. Overall, the applied pipeline achieved an effective balance between dimensionality reduction and information preservation.

Snippet from the code:

```
Input shape: (1661, 6686)
Xs shape: (1661, 6628)
Feature names length: 6628
MI reduced shape: (1661, 100)
PCA 90% components: 657
PCA 95% components: 894
PCA reduced shape: (1661, 894)
CV results: {'raw': np.float64(
Feature selection completed.
```

Filip Brutovský
Anastasiia Rerikh

Neural Network Overview

Model Architectures:

This project implements four distinct neural network architectures for comparative analysis.

Model 0: SimpleCNN (SCNN)

- Input: 3×224×224
- Conv1: 3→32, kernel 3×3, output 32×224×224, MaxPool → 32×112×112
- Conv2: 32→64, kernel 3×3, output 64×112×112, MaxPool → 64×56×56
- Conv3: 64→128, kernel 3×3, output 128×56×56, MaxPool → 128×28×28
- Flatten: 128·28·28 = 100,352 features
- FC1: 100,352 → 256, dropout applied
- FC2: 256 → num_classes

Model 1: DeepCNN (DCNN)

- Input: 3×224×224
- Conv1: 3→32, BN, GELU, MaxPool → 32×112×112
- Conv2: 32→64, BN, GELU, MaxPool → 64×56×56
- Conv3: 64→128, BN, GELU → 128×56×56
- Conv4: 128→128, BN, ReLU, MaxPool → 128×28×28
- Conv5: 128→256, BN, GELU → 256×28×28
- GAP: AdaptiveAvgPool → 256×16×16
- Flatten: 256·16·16 = 65,536 features
- FC1: 65,536 → 512, GELU, dropout applied
- FC2: 512 → num_classes

Model 2: ResNet

- Input: 3×224×224
- Block1: 3→32, two 3×3 convs + BN + residual 1×1, output 32×224×224, MaxPool → 32×112×112
- Block2: 32→64, two 3×3 convs + BN + residual, MaxPool → 64×56×56
- Block3: 64→128, same structure, MaxPool → 128×28×28
- Block4: 128→256, same structure, MaxPool → 256×14×14
- Flatten: 256·14·14 = 50,176 features
- FC: 50,176 → num_classes

Model 3: FENN (Feature Extraction Neural Network)

- Input: 101-dim feature vector
- Layer1: 101 → 512, BN + Dropout + LeakyReLU
- Layer2: 512 → 1024, BN + Dropout + LeakyReLU
- Layer3: 1024 → 512, BN + Dropout + LeakyReLU
- Layer4: 512 → 256, BN + Dropout + LeakyReLU
- Output: 256 → num_classes (14)

Short Architecture Comparison

Model	**Type**	**Conv Layers**	***FC Layers**	**Regularisation**	**Key Feature**
SCNN	CNN	3	2	Dropout	Baseline simplicity
DCNN	CNN	5	2	BatchNorm + Dropout	Depth and Global Average Pooling (GAP)
ResNet	CNN	8 (4blocks×2)	1	BatchNorm + Dropout	Skip connections
FENN	MLP	0	4	BatchNorm + Dropout	Feature engineering

**Fully connected Layer/Dense Layer*

Training Configuration

Hyperparameters:

Global Configuration

Device: CPU/CUDA (if available)

Batch size: 32/64

Number of epochs: 5/10/15/20/25/30/50/100

Random state: 14

Optimisation Parameters

Optimiser: AdamW

Learning rate: 0.0005

Weight decay: 0.0005 (5e-4)

Loss Function: CrossEntropyLoss

Regularisation Parameters

Dropout: 0.2/0.4 (Applied to: SCNN, DCNN, FENN)

Batch Normalisation is applied to DCNN, ResNet, FENN

Weight Decay: 5e-4 (all networks)

Training Modes:

Simple Training (Mode 1)

Uses a training set only, with no validation monitoring during training, for faster experiments. Scheduler is triggered by training loss.

Complex Training (Mode 0)

Uses both training and validation sets, where validation metrics are computed after each epoch (validation monitoring). Scheduler triggered by validation loss, which is better generalisation.

Metrics Logged:

- Training loss and accuracy per epoch
- Validation loss and accuracy per epoch

- Learning rate updates
- Best model checkpoint based on validation performance

Training Pipeline:

1. Initialization
    a. Load configuration from *config.yaml*
    b. Set random seeds for reproducibility
    c. Initialise model architecture
    d. Move model to GPU/CPU
2. Data Loading
    a. Load preprocessed data/features
    b. Create DataLoader instances
    c. Apply augmentation to the training set
3. Training Loop (per epoch)
    a. Set the model to training mode
    b. Iterate through training batches:
        i. Forward pass
        ii. Compute loss
        iii. Backward pass
        iv. Update weights
    c. Compute epoch training metrics
4. Validation Loop (per epoch)
    a. Set the model to evaluation mode
    b. Iterate through validation batches:
        i. Forward pass (no gradients)
        ii. Compute loss
    c. Compute epoch validation metrics
    d. Update learning rate if needed
5. Model Saving
6. Final Evaluation
    a. Load the best model
    b. Evaluate on the test set
    c. Generate a confusion matrix and metrics

Experiments

The goal of our experiments is to compare four architectures and identify the best-performing model for sports classification. Our evaluation metrics are: **accuracy**, **precision**, **recall** and **F1-Score**. All metrics, except for accuracy, are weighted because our dataset has a mild class imbalance (1.64:1 ratio), so we need to account for different class sizes, giving more importance to larger classes while still considering smaller ones.

Concluded Experiments:

We concluded countless experiments with types of the model (SCNN, DCNN, ResNet, FENN).

- SCNN: Proved to be too simple to capture complex information from the images.
- DCNN: Experimented with depth and found the optimal depth for image recognition.
- ResNet: Most time and computation power-expensive, and proved to be too complex for our cause and not worth the time and power.
- FENN: We experimented with depth (3,4,5,6) FC layers. And after the experiments, we concluded that 4 FC layers are optimal.

Experiments with optimisers.

- Adam: After implementing improved efficiency but it still lacked consistency and better synergy with weight decay.
- AdamW: Better results than Adam and proved to be more stable.

Experiments with scheduler:

- None: The constant LR model had issues with overfitting.
- ReduceLROnPlateau: Improved metrics after implementation, but only part.
- CosineAnnealingLR: Caused quicker learning but stagnated after a while.

After these experiments, we concluded that we will use DCNN with AdamW and ReduceLROnPlateau, because they proved to be the best.

The second phase of experiments was more about hyperparameter tuning, and we split the experiments into experiments with FENN and DCNN:

DCNN:

- Transformations:
  - horizontal_flip_prob: 0.3/0.4/0.5
  - random_rotation_degree: 30/20/50
  - color_jitter:
    - brightness: 0.2/0.3
    - contrast: 0.2/0.3
    - saturation: 0.2/0.3
    - hue: 0.1/0.2/0.3
- Hyperparameters:
  - batch_size: 32/64/128
  - epochs: 20/30/40
  - dropout: 0.2/0.3/0.4
- Optimizer:
  - learning_rate: 0.001/0.0005
  - weight_decay: 1e-4/5e-4/1e-3/1e-2
- Scheduler:
  - mode: min/max
  - factor: 0.5
  - patience: 2

- Transformations:
  - None

    Because the feature extraction wouldn't allow it, and when we tried the transformation, it plummeted the metrics.
- Hyperparameters:
  - batch_size: 32/64/128
  - epochs: 50/100/150
  - dropout: 0.3/0.4/0.5
- Optimizer:
  - learning_rate: 0.001/0.0005/0.00005
  - weight_decay: 1e-4/5e-4/1e-3/1e-2
- Scheduler:
  - mode: min/max
  - factor: 0.5
  - patience: 2

Filip Brutovský
Anastasiia Rerikh

Results and evaluation

Best Network Performance

Name of the network: revived-sky-777_DCNN

Training completed in: 7m 31s

Best epoch: 16 and 23

Final Validation Metrics (for the best)

Accuracy: 0.90625

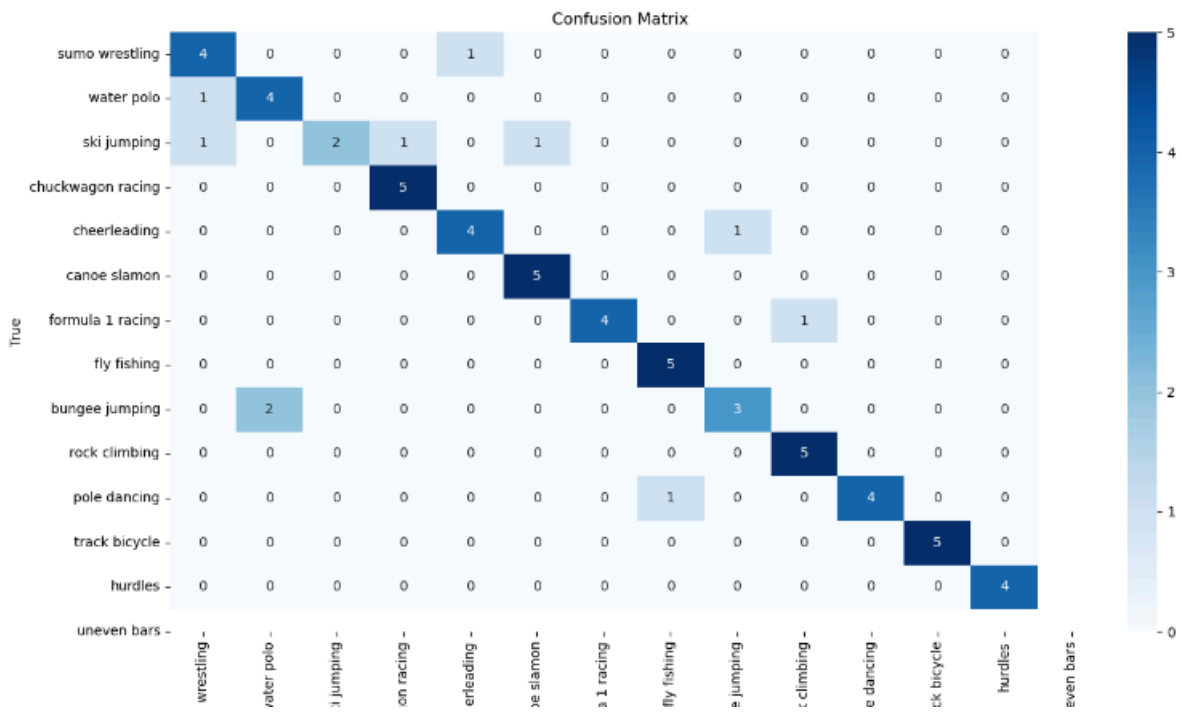
Precision: 0.91309

Recall: 0.90625

F1-Score: 0.89855

Confusion Matrix

Interpretation:



Filip Brutovský
Anastasiia Rerikh

Best FENN Performance

Name of the network: giddy-grass-796_FENN

Training completed in: 25s

Best epoch: 99

Final Validation Metrics (for the best)

Accuracy: 0.8

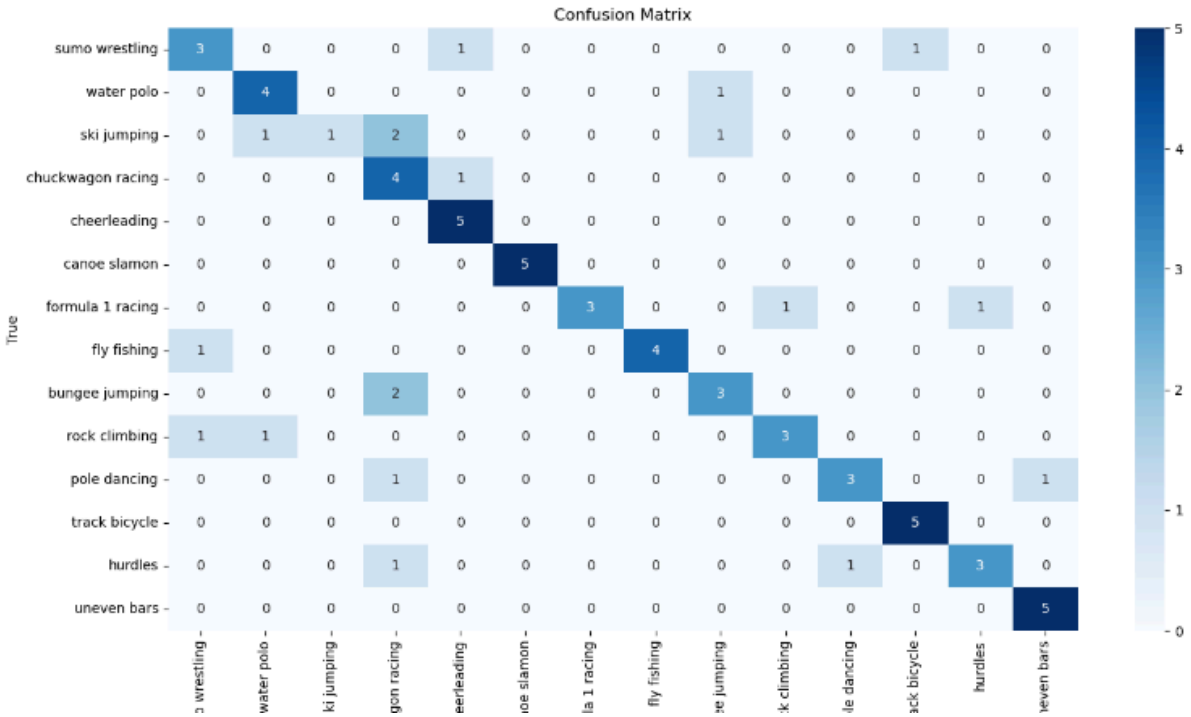
Precision: 0.83934

Recall: 0.8

F1-Score: 0.79234

Confusion Matrix

Interpretation:

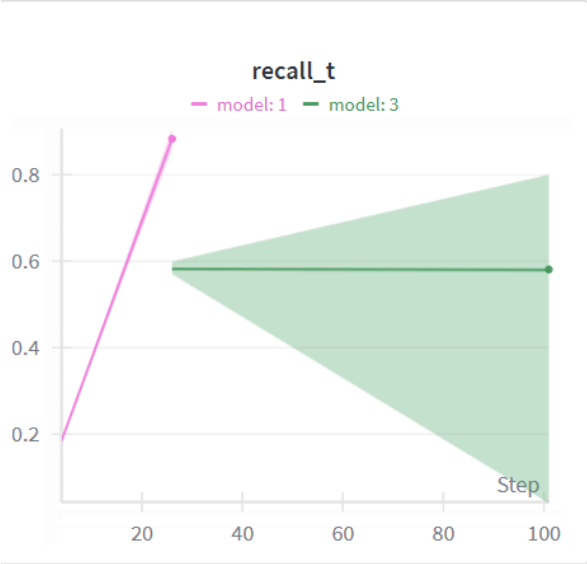


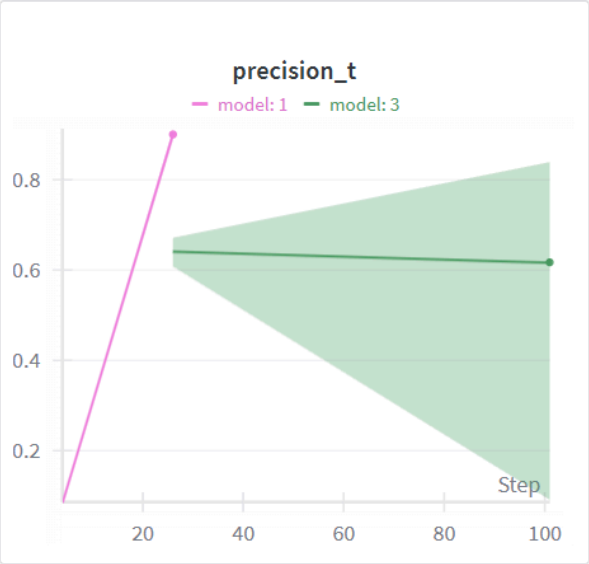
Filip Brutovský
Anastasiia Rerikh

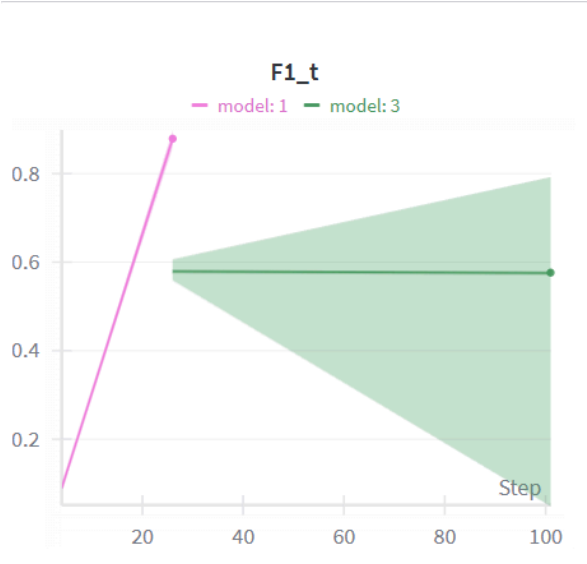
Comparative Analysis

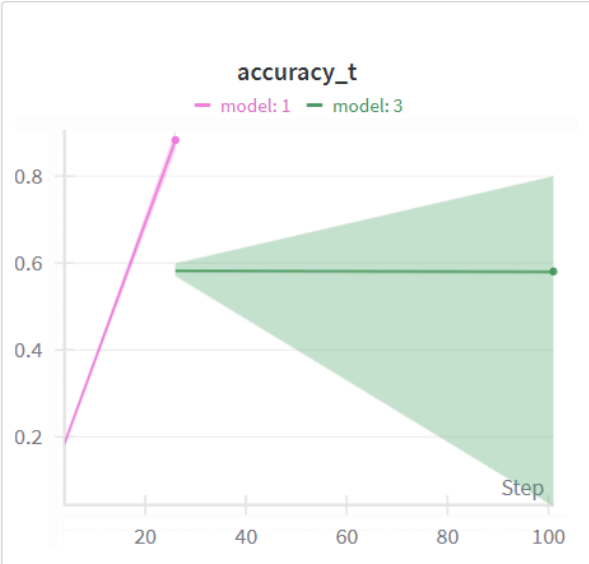
Graphs

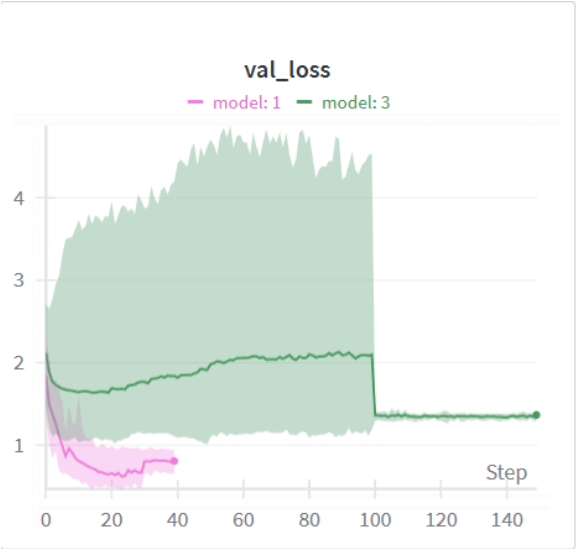
model 1: DCNN | model 3: FENN

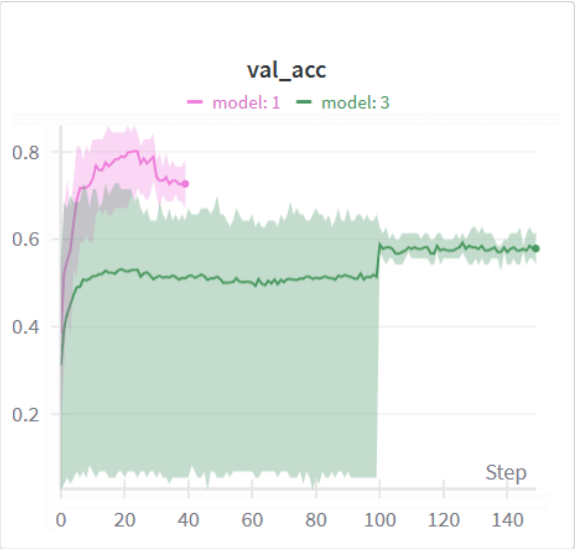


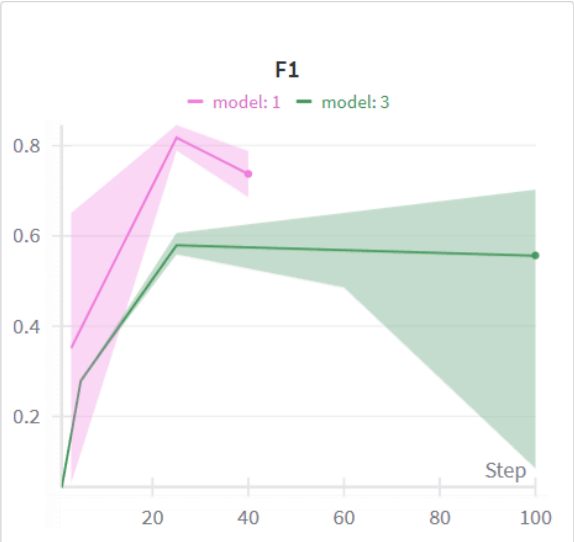






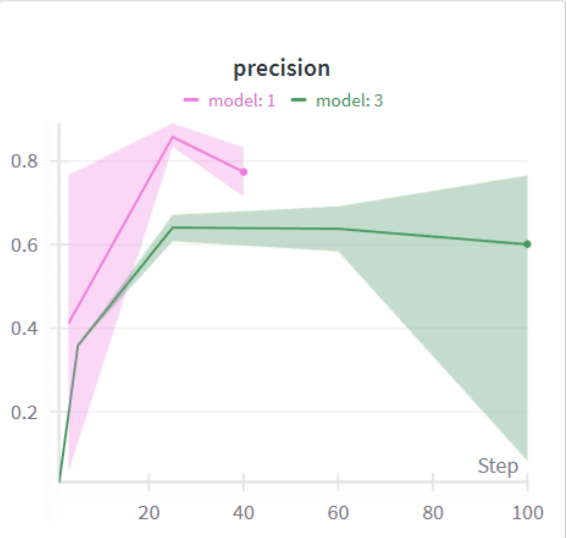

val_loss
model: 1
model: 3
4
3
2
1
Step
0
20
40
60
80
100
120
140

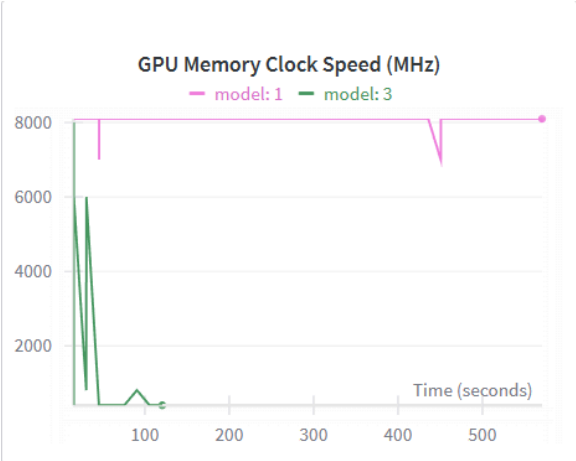

val_acc
model: 1
model: 3
0.8
0.6
0.4
0.2
Step
0
20
40
60
80
100
120
140

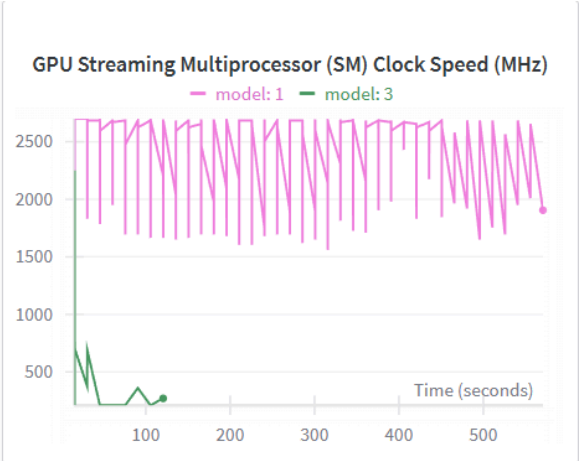

F1
model: 1
model: 3
0.8
0.6
0.4
0.2
Step
20
40
60
80
100

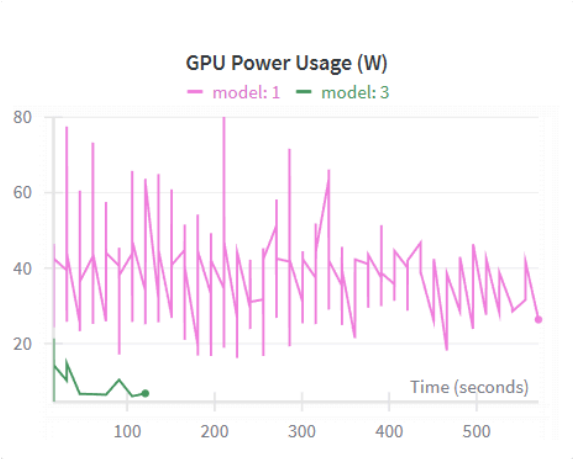
recall
model: 1
model: 3
0.8
0.6
0.4
0.2
Step
20
40
60
80
100

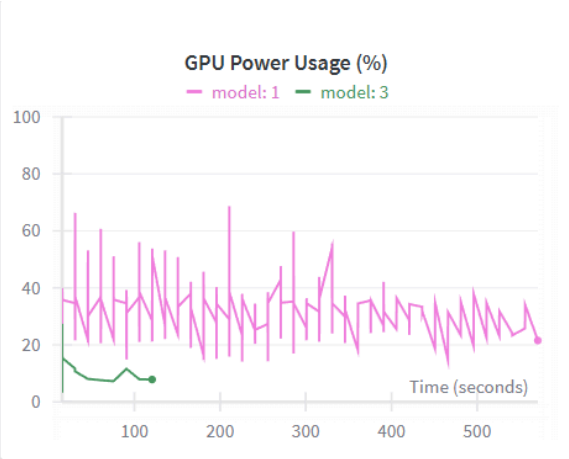
accuracy
model: 1
model: 3
0.8
0.6
0.4
0.2
Step
20
40
60
80
100

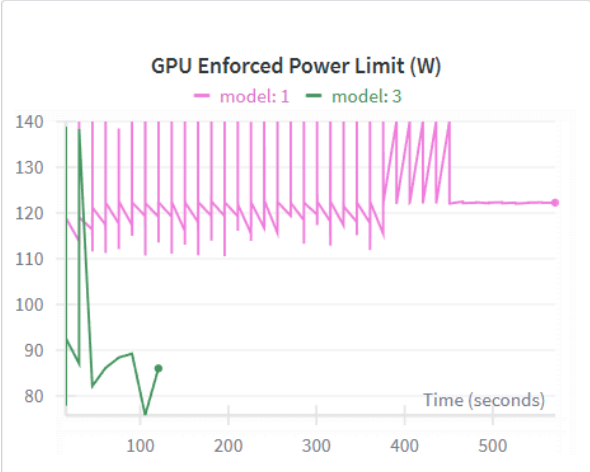

precision
model: 1
model: 3
0.8
0.6
0.4
0.2
Step
20
40
60
80
100

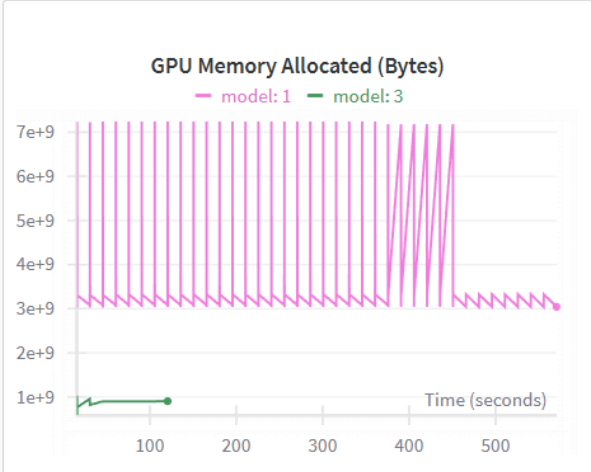

GPU Memory Clock Speed (MHz)
model: 1
model: 3
8000
6000
4000
2000
Time (seconds)
100
200
300
400
500


GPU Streaming Multiprocessor (SM) Clock Speed (MHz)
model: 1
model: 3
2500
2000
1500
1000
500
Time (seconds)
100
200
300
400
500


GPU Power Usage (W)
model: 1
model: 3
80
60
40
20
Time (seconds)
100
200
300
400
500


GPU Power Usage (%)
model: 1
model: 3
100
80
60
40
20
0
Time (seconds)
100
200
300
400
500


GPU Enforced Power Limit (W)
model: 1
model: 3
140
130
120
110
100
90
80
Time (seconds)
100
200
300
400
500


GPU Memory Allocated (Bytes)
model: 1
model: 3
7e+9
6e+9
5e+9
4e+9
3e+9
2e+9
1e+9
Time (seconds)
100
200
300
400
500

Filip Brutovský
Anastasiia Rerikh

Experimental conclusion

The goal of our experiments was to compare four architectures—SCNN, DCNN, ResNet and FENN—and determine which model performs best on our sports classification task. We evaluated accuracy along with weighted precision, recall and F1-score. The experiments showed clear differences between the models. SCNN was too shallow to capture the complexity of the images, while ResNet proved unnecessarily heavy for our data and offered no practical benefit compared to its long training time. FENN performed reasonably well with four fully connected layers, but its reliance on hand-crafted features limited its ability to generalise, and image transformations could not be used without damaging the extracted features. DCNN provided the best balance of depth, performance and training efficiency, and further tuning confirmed it as the strongest candidate.

In optimiser experiments, AdamW consistently produced more stable training and better generalisation than Adam. For schedulers, keeping the learning rate constant led to overfitting, CosineAnnealingLR accelerated early learning but stagnated, and ReduceLROnPlateau delivered the most stable improvements when progress slowed. Combining these findings, we selected the DCNN architecture with AdamW and ReduceLROnPlateau as the optimal setup.

To compare the performance of neural network with manually extracted features and traditional CNN we concluded that overall CNN was more accurate but to train such a model requires abundance of time and computing power as when compared to FENN we managed to squeeze out approximately 80% accuracy on test data and the training time (25s after dataframe augmentations) in comparison to roughly 7 minutes (that isn't that much too) is just a fraction.

The final model, named revived-sky-777_DCNN, trained in seven and a half minutes and achieved strong validation performance, with accuracy of 0.90625, precision of 0.91309, recall of 0.90625 and an F1-score of 0.89855. These results confirm that a moderately deep CNN with proper regularisation and adaptive scheduling offers the best balance of accuracy, stability and computational efficiency for this classification problem.

Overall Conclusion

To conclude our project we tried to develop a NN that will classify 14 categories of sports. We experimented with two methods, traditional convolutional networks and with outdated methods of manually extracting the features. In the experimental phase we found out that CNN had better performance metrics averaging roughly 90% and FENN had worse metrics but the training speed and cost were significantly lower than CNN. We concluded that with rapid development of technology and hardware performance it is not surprising that less accurate manual feature extractions are no longer needed.